

สรุปเปรียบเทียบ Algorithm A และ Algorithm B

หัวข้อ	Algorithm A: FIFO Queue	Algorithm B: Priority Queue
โครงสร้างข้อมูล	ArrayDeque	PriorityQueue
หลักการทำงาน	First In First Out	Priority สูงออกก่อน
การจัดลำดับ	ตามเวลาที่เข้ามา	ตามระดับความรุนแรง
Event ที่มี Priority สูง	ไม่ได้รับสิทธิ์พิเศษ	ได้รับการประมวลผลก่อน
กรณี Priority เท่ากัน	ไม่เกี่ยวข้อง	ใช้ลำดับการเข้ามา (Sequence)
Enqueue	$O(1)$	$O(\log n)$
Dequeue	$O(1)$	$O(\log n)$
Peek	$O(1)$	$O(1)$
Search	$O(n)$	$O(n)$
Display	$O(n)$	$O(n)$
Space Complexity	$O(n)$	$O(n)$
ข้อดี	ทำงานง่าย รวดเร็ว และยุติธรรมตามลำดับเวลา	สามารถจัดการเหตุการณ์ที่สำคัญก่อน
ข้อจำกัด	เหตุร้ายแรงอาจต้องรอเหตุที่มาก่อน	เหตุ Priority ต่ำอาจต้องรอนาน
เหมาะกับ	งานที่ต้องประมวลผลตามลำดับการเข้ามา	ระบบฉุกเฉินที่ต้องให้ความสำคัญกับเหตุรุนแรง

ตัวอย่างจากโครงการ

Algorithm A: FIFO

$E1 \rightarrow E2 \rightarrow E3 \rightarrow E4 \rightarrow E5 \rightarrow E6$

ประมวลผลตามลำดับเวลาที่รับแจ้ง โดยไม่สนใจระดับความรุนแรง

Algorithm B: Priority Queue

E3 → E5 → E4 → E1 → E6 → E2

ประมวลผลตามระดับความรุนแรง

Critical → High → Medium → Low

และหากมีระดับเดียวกัน จะเลือก Event ที่เข้ามาก่อน

สรุป

Algorithm A (FIFO Queue) เหมาะสำหรับระบบที่ต้องการความยุติธรรมตามลำดับเวลาการเข้ามาของข้อมูล และมีประสิทธิภาพสูงในการ enqueue และ dequeue เนื่องจากมี Time Complexity เท่ากับ $O(1)$ ส่วน **Algorithm B (Priority Queue)** เหมาะกับระบบ Emergency Call Dispatch เนื่องจากสามารถให้เหตุการณ์ที่มีความรุนแรงสูงได้รับการประมวลผลก่อน แม้จะเข้ามาทีหลัง แต่มีค่าใช้จ่ายในการ enqueue และ dequeue สูงกว่า FIFO เป็น $O(\log n)$ ดังนั้น สำหรับระบบแจ้งเหตุฉุกเฉิน **Priority Queue มีความเหมาะสมมากกว่า** เพราะความสำคัญของเหตุการณ์ควรถูกนำมาพิจารณาร่วมกับลำดับเวลาในการรับแจ้งเหตุ